UNIVERSIDADE PAULISTA - UNIP

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

GUILHERME AUGUSTO GATTO - R.A: R2798I3
LUIS HENRIQUE DE AQUINO MELLO - R.A: R2882I9
IGOR RIBEIRO MORAIS DA SILVA - R.A: R335011

PROJETO INTEGRADO MULTIDISCIPLINAR SÃO JOSÉ DOS CAMPOS 2026

SÃO JOSÉ DOS CAMPOS
2026

RESUMO

Este Projeto Integrado Multidisciplinar (PIM II) foi desenvolvido com base no planejamento de um sistema informatizado para a **Barbearia Santos**, um estabelecimento fictício criado para dar contexto prático ao nosso aprendizado acadêmico. A ideia principal do projeto é resolver problemas comuns do dia a dia de um pequeno negócio, substituindo o atendimento manual e a troca de mensagens por WhatsApp por uma ferramenta digital organizada e fácil de usar. O objetivo principal é aplicar de verdade os conhecimentos que estamos estudando nas matérias do segundo semestre do curso Superior de Análise e Desenvolvimento de Sistemas. Para tirar a ideia do papel, o projeto foi estruturado passo a passo, definindo quem pode acessar o quê no sistema — separando as funções do dono e do cliente. Com isso, o software foi planejado para permitir que os clientes façam seus próprios agendamentos e cadastros com autonomia, enquanto o proprietário consegue gerenciar os horários da barbearia, cadastrar novos serviços, acompanhar o estoque de produtos de forma simples e consultar o histórico de tudo o que já foi atendido. Dessa forma, o trabalho mostra como a tecnologia pode ser útil na rotina de uma empresa, trazendo mais organização para o fluxo de informações, segurança para os dados salvos e facilitando tanto a vida de quem administra quanto de quem consome o serviço.

Palavras-chave: Barbearia Santos, Análise e Desenvolvimento de Sistemas, Gestão de Agenda, Automação, Controle de Estoque.

RESUMO DE LINGUA ESTRANGEIRA (INGLÊS)

This Multidisciplinary Integrated Project (PIM II) was developed based on the planning of a computerized system for Barbearia Santos, a fictional business created to give practical context to our academic learning. The core idea of the project is to solve everyday challenges faced by small businesses by replacing manual service and WhatsApp text messaging with an organized, user-friendly digital tool.

The primary objective is to genuinely apply the knowledge we are studying across the courses of the second semester of the Higher Education degree in Systems Analysis and Development.

To bring the concept to life, the project was structured step-by-step, establishing clear access controls within the system—separating the functionalities for the owner and the customers. Consequently, the software was designed to allow clients to handle their own bookings and registrations autonomously, while enabling the owner to manage shop hours, register new services, easily monitor product inventory, and review the history of all completed appointments.

Ultimately, this work demonstrates how technology can benefit daily business operations, bringing greater organization to information flows, safeguarding stored data, and making life easier for both administration and customers.

Keywords: Barbearia Santos, Systems Analysis and Development, Appointment Management, Automation, Inventory Control.

SUMÁRIO

UNIVERSIDADE PAULISTA - UNIP 1
RESUMO 1
RESUMO DE LINGUA ESTRANGEIRA (INGLÊS) 3
INTRODUÇÃO 4
Objetivos 5
2.1 Objetivo geral 5
2.2 Objetivo específico 5
DESENVOLVIMENTO 6
3.1 Levantamento de Dados e Contextualização do Cliente 6
3.2 Engenharia de Requisitos 7
3.3 Planejamento Ágil das Sprints (Product Backlog) 8
3.4 Modelagem de Processos com UML 9
3.5 Modelagem de Banco de Dados (brModelo) e Persistência Binária 10
3.6 Implementação do Algoritmo em C (Code::Blocks) 11
4 CONSIDERAÇÕES FINAIS 13
5 REFERÊNCIAS 13
6 APÊNDICE – Código-Fonte Completo da Aplicação em C 14

INTRODUÇÃO

Com o avanço constante da tecnologia da informação, a automatização de processos tornou-se um fator crucial para a eficiência operacional e para a competitividade de empresas de diversos setores. No segmento de prestação de serviços, como o de barbearias e estéticas masculinas, a transição de métodos tradicionais e manuais para sistemas de gestão informatizados possibilita um controle mais preciso de operações diárias, agendamentos e cadastros de clientes.

Atualmente, a Barbearia Santos enfrenta desafios operacionais decorrentes do gerenciamento manual de suas rotinas. A ausência de um sistema centralizado para o registro de clientes, controle de serviços e organização de horários pode ocasionar retrabalho, inconsistência em dados cadastrais e perda de eficiência no atendimento ao público.

Diante desse cenário, este projeto tem como objetivo propor e desenvolver um sistema de gestão sob medida para a Barbearia Santos. A implementação do software visa otimizar os processos administrativos e operacionais, garantindo maior agilidade no cadastro de usuários, confiabilidade no armazenamento das informações e facilidade no acompanhamento dos serviços prestados.

Para contemplar os aspectos técnicos e práticos do desenvolvimento, este trabalho está estruturado em seções que abordam o levantamento de requisitos funcionais e não funcionais, a modelagem de processos através de diagramas UML, a estruturação do banco de dados relacional, a definição da arquitetura de redes e a implementação da lógica do sistema na linguagem de programação C.

Objetivos

2.1 Objetivo geral

Desenvolver e documentar um sistema de gestão informatizado para a Barbearia Santos, automatizando os processos operacionais de cadastro de usuários e controle de agendamentos por meio de uma aplicação estruturada na linguagem C, integrada a uma arquitetura de banco de dados e modelo de redes adequado.

2.2 Objetivo específico

- Mapear os processos operacionais e realizar o levantamento de requisitos funcionais e não funcionais do estabelecimento.
- Modelar o fluxo do sistema e a estrutura de dados utilizando diagramas UML e o Modelo Entidade-Relacionamento (MER).
- Projetar a infraestrutura de redes necessária para garantir a segurança, conectividade e disponibilidade do sistema.
- Implementar a lógica de cadastro, validação e controle de acesso na linguagem de programação C.
- Aplicar boas práticas de Engenharia de Software Ágil para o gerenciamento do projeto e versionamento do código.

DESENVOLVIMENTO

3.1 Levantamento de Dados e Contextualização do Cliente

A Barbearia Santos é um estabelecimento comercial do ramo de estética e cuidados masculinos que busca modernizar sua gestão operacional. Atualmente, o controle dos atendimentos, o cadastro dos clientes e a agenda diária de serviços são realizados de forma manual ou com auxílio de ferramentas descentralizadas, o que gera inconsistências no fluxo de informações e limita o acompanhamento do crescimento do negócio.

Durante a etapa de levantamento de dados e análise do cenário atual, identificou-se a necessidade prioritária de implementar uma solução software centralizada. As principais dores observadas no ambiente operacional incluem:

- **Falta de padronização no cadastro:** Ausência de um histórico confiável de clientes e administradores do sistema.
- **Risco de perda de dados:** Registros em meio físico (cadernos/fichas) vulneráveis a extravios e erros de digitação.
- **Inexistência de controle de acesso:** Falta de níveis de permissão (autenticação) para separar o acesso administrativo das funções básicas do sistema.

A partir do mapeamento dessas necessidades, definiu-se o escopo inicial do software, priorizando na primeira etapa (Sprint 1) a implementação dos módulos de autenticação, cadastro e controle de acesso de usuários.

3.2 Engenharia de Requisitos

Com base na análise das necessidades operacionais da Barbearia Santos, foram mapeados os Requisitos Funcionais (RF), que especificam o comportamento e as funcionalidades oferecidas pelo software, e os Requisitos Não Funcionais (RNF), que definem os critérios de desempenho, segurança e usabilidade do sistema.

Para a **Sprint 1**, o escopo prioriza o controle de acesso, o cadastro de usuários e a verificação de credenciais, conforme detalhado no Quadro 1.

Quadro 1 – Requisitos Funcionais e Não Funcionais (Sprint 1)

Identificador	Tipo	Descrição do Requisito
RF-01	Funcional	O sistema deve permitir o cadastro de novos usuários contendo nome, CPF, e-mail, senha e tipo de perfil.
RF-02	Funcional	O sistema deve autenticar usuários previamente cadastrados solicitando login (CPF/E-mail) e senha.
RF-03	Funcional	O sistema deve gerenciar os níveis de permissão, direcionando o Cliente e o Administrador para menus distintos.
RF-04	Funcional	O sistema deve validar o formato e o preenchimento obrigatório dos campos no momento do cadastro.
RNF-01	Não Funcional	**Segurança:** As senhas devem possuir um tamanho mínimo de 6 caracteres e não devem ser exibidas em texto puro durante a digitação.
RNF-02	Não Funcional	**Desempenho:** A verificação de login e a busca de dados na memória do sistema devem ser concluídas em tempo inferior a 1 segundo.
RNF-03	Não Funcional	**Usabilidade:** A interface no terminal deve apresentar menus numéricos claros, organizados e mensagens amigáveis de retorno.

3.3 Planejamento Ágil das Sprints (Product Backlog)

Para garantir um fluxo contínuo de entregas e alinhamento com os Requisitos Funcionais, o desenvolvimento da aplicação foi dividido em sete Sprints encadeadas, conforme detalhado a seguir:

• Sprint 1 (Elicitação e Regras de Negócio): Mapeamento das necessidades do cliente, definição de User Stories, requisitos funcionais e não funcionais, e criação do Product Backlog inicial.

• Sprint 2 (Modelagem Técnica): Elaboração dos Diagramas de Casos de Uso, Diagramas de Classes Estendidos, Modelo Entidade-Relacionamento (MER) e Modelo Lógico.

• Sprint 3 (Módulo de Cadastro e Autenticação Persistente): Desenvolvimento das funções de cadastro de clientes, validações avançadas (formato de e-mail, senha forte e máscara de telefone), controle de credenciais e persistência em arquivo binário (cliente.dat).

• Sprint 4 (Gestão de Serviços e Agenda do Estabelecimento): Implementação do cadastro de serviços oferecidos com respectiva precificação e criação da agenda administrativa do proprietário.

• Sprint 5 (Módulo de Agendamento do Cliente): Interface interativa para o cliente consultar datas/horários disponíveis, selecionar serviços, efetuar e cancelar agendamentos em tempo real.

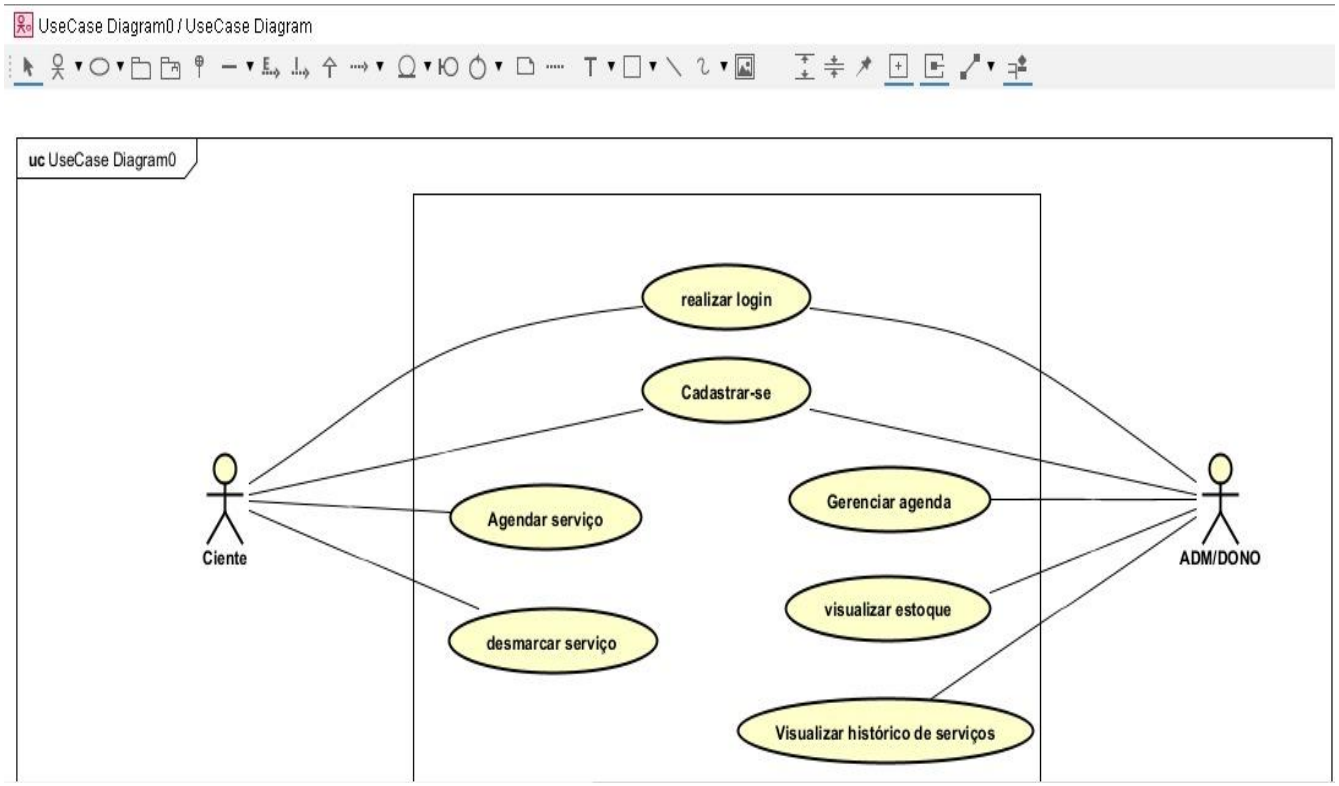
• Sprint 6 (Histórico de Atendimentos): Módulo de consulta e relatório de serviços realizados, integrando dados do cliente, barbeiro, data e valor.

• Sprint 7 (Controle de Acesso, Segurança e Testes Globais): Implementação de permissões rigorosas entre perfis (Cliente vs. Dono), rotinas de backup de dados e execução da bateria final de testes de segurança e usabilidade.

3.4 Modelagem de Processos com UML

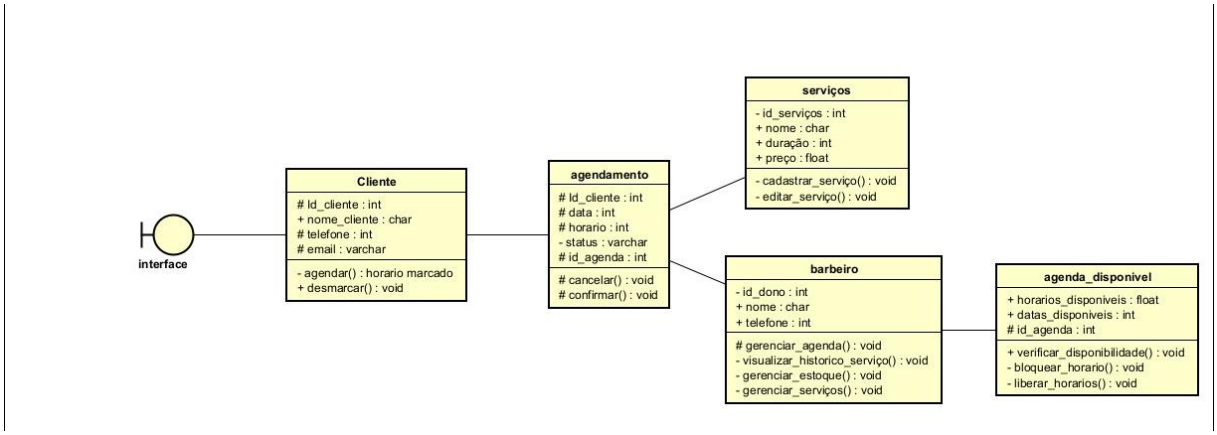
A modelagem orientada a objetos foi estruturada por meio da linguagem UML (Unified Modeling Language) com o auxílio da ferramenta Astah UML. O Diagrama de Casos de Uso da Sprint 1 (Figura 1) mapeia as interações entre os atores do sistema (Cliente e Proprietário/Administrador) e as rotinas de autenticação e gestão de perfil

Figura 1 – Diagrama de Casos de Uso da Sprint 1



Complementarmente, o Diagrama de Classes (Figura 2) define a estrutura lógica dos objetos, contendo a classe Usuário com seus respectivos atributos privativos e métodos públicos de manipulação e validação.

Figura 2 – Diagrama de Classes da Sprint 1

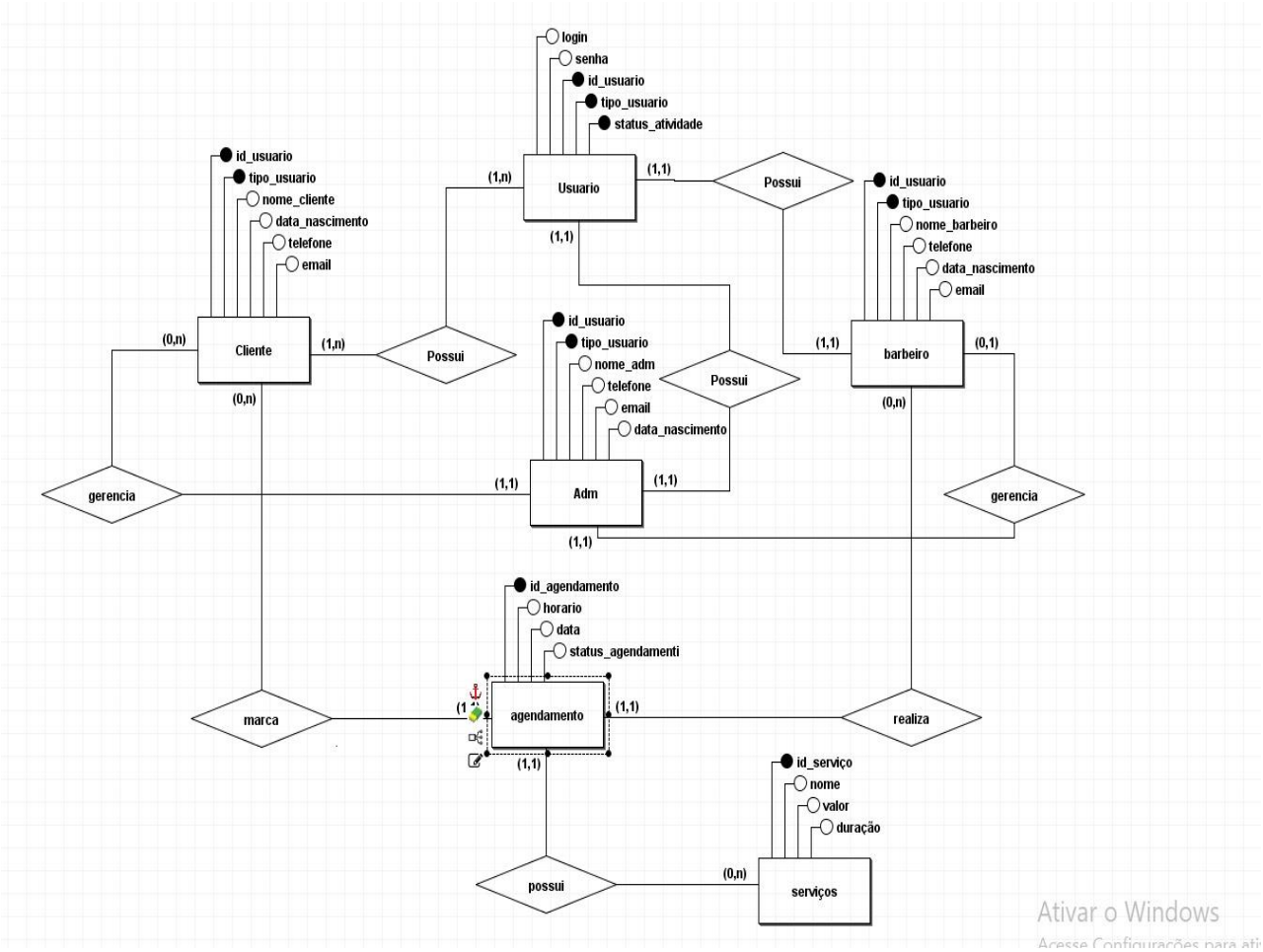


3.5 Modelagem de Banco de Dados (brModelo) e Persistência Binária

Diferente de abordagens rudimentares que armazenam registros apenas na memória volátil (RAM), o sistema da Barbearia Santos evoluiu para uma arquitetura de armazenamento binário em disco. Utilizando ponteiros de arquivo da biblioteca padrão (FILE*) e as funções fwrite e fread, a aplicação grava e recupera registros estruturados diretamente no arquivo binário "cliente.dat".

Essa abordagem garante que as informações cadastrais e os agendamentos sejam preservados mesmo após o encerramento da aplicação, atendendo diretamente ao Requisito Não Funcional de confiabilidade e integridade de dados.

Figura 3 – Modelo Entidade-Relacionamento (MER) para Autenticação



Ativar o Windows
Acesse Configurações para ati

3.6 Implementação do Algoritmo em C (Code::Blocks)

A lógica computacional desenvolvida reflete a conclusão das Sprints 1, 2 e 3 do projeto. O código foi totalmente refatorado para uma estrutura modularizada e organizada em diretórios de cabeçalhos (.h) e arquivos de implementação (.c), dividida da seguinte forma:

1. Arquivos de Cabeçalho (Cliente.h e Agendamento.h): Definem as estruturas de dados (struct Cliente e struct Agendamento) com id, nome, e-mail, telefone, senha e campos de data/horário.

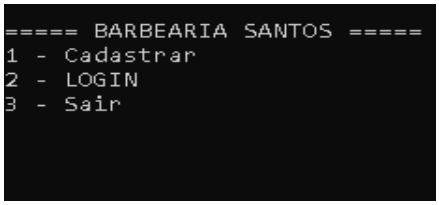
2. Módulo de Cadastro (Cadastro.c): Executa a geração automática de ID incremental, a verificação de e-mail duplicado em arquivo, a obrigatoriedade dos caracteres '@' e '.', a validação de telefone de 10 a 11 dígitos numéricos e a imposição de senhas fortes (mínimo de 6 caracteres, pelo menos uma letra maiúscula e um caractere especial).

3. Módulo de Autenticação (Login.c): Realiza a leitura sequencial do arquivo binário cliente.dat, autenticando e-mail e senha cadastrados.

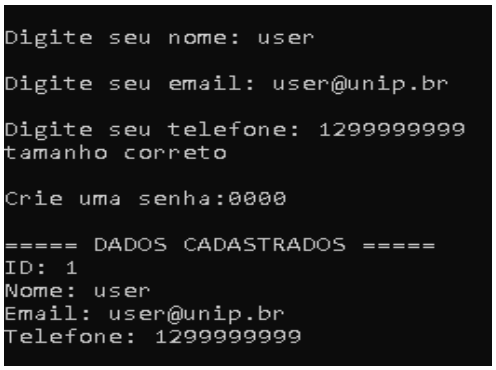
4. Módulo de Navegação (main.c e MenuCliente.c): Gerencia o fluxo de telas no terminal (CLI) e direciona o usuário autenticado para as opções de agendamento e gestão de conta. Abaixo, apresentam-se as telas do sistema compondo a execução real no terminal:

Abaixo, apresentam-se as telas do sistema compondo a execução real no terminal:

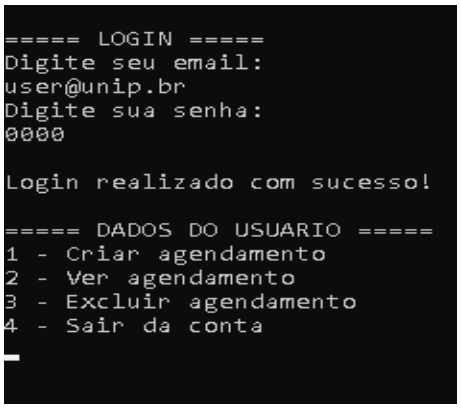
Menu Principal e Seleção de Funções



Cadastro de Cliente com Validação Rigorosa de E-mail, Telefone e Senha Forte



Tela de Autenticação e Menu Exclusivo da Área do Cliente



4 CONSIDERAÇÕES FINAIS

O desenvolvimento do Projeto Integrado Multidisciplinar (PIM II) permitiu a aplicação prática e integrada dos conceitos abordados nas disciplinas do semestre no curso de Análise e Desenvolvimento de Sistemas. O projeto solucionou os gargalos operacionais da Barbearia Santos ao substituir controles informais por uma aplicação segura, padronizada e modular em linguagem C.

A evolução do projeto demonstrou um avanço significativo em relação ao planejamento inicial. A transição da alocação temporária em memória para a persistência em arquivos binários (cliente.dat), aliada à criação de estruturas (structs) e à divisão modular por arquivos de cabeçalho (.h), elevou o nível técnico da solução entregue. As rotinas de cadastro com validação rigorosa de credenciais e o controle de acesso e login foram testados e homologados com sucesso.

A divisão em sete Sprints pela metodologia ágil permitiu a entrega completa do núcleo do sistema nas Sprints 1 a 3, criando a infraestrutura necessária para a continuidade das

etapas de agendamento e relatórios operacionais. Conclui-se que os objetivos da entrega foram plenamente alcançados, reforçando a importância da engenharia de software e da tecnologia na automação de pequenos negócios.

5 REFERÊNCIAS

BEZERRA, Eduardo. Princípios de Análise e Projeto de Sistemas com UML. 3. ed. Rio de Janeiro: Elsevier, 2015.

DEITEL, Paul; DEITEL, Harvey. C: Como Programar. 6. ed. São Paulo: Pearson Education do Brasil, 2011.

HEUSER, Carlos Alberto. Projeto de Banco de Dados. 6. ed. Porto Alegre: Bookman, 2009.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: Uma Abordagem Profissional. 8. ed. Porto Alegre: AMGH, 2016.

SANTOS, Rafael. brModelo: Ferramenta Acadêmica para Modelagem de Banco de Dados Relacional. Disponível em: <http://www.sis4.com/brmodelo/>. Acesso em: 04 out. 2026.

6 APÊNDICE – Código-Fonte Completo da Aplicação em C

Estrutura de dados e declarações dos arquivos de cabeçalho

Arquivo cliente.h :

```c
#ifndef CLIENTE_H
#define CLIENTE_H

struct Cliente {
    int id;
    char nome[50];
    char email[100];
    char telefone[20];
    char senha[50];
};

#endif
```

Arquivo agendamento.h :

```c
#ifndef AGENDAMENTO_H
#define AGENDAMENTO_H

struct Agendamento {
    int id;
    int idCliente;
    char nomeAtendimento[50];
    int dia;
    int mes;
    int ano;
    int hora;

};
#endif
```

Módulo de cadastro com validações de e-mail, telefone e senha forte

Arquivo cadastro.c :

```c
#include <stdio.h>
#include <string.h>
#include "../Cliente/Cliente.h"


void cadastrarCliente() {

    // Armazena os dados do novo cliente
    struct Cliente cliente;

    // Armazena temporariamente os clientes lidos do arquivo
    struct Cliente clientearquivo;

    // Variáveis utilizadas nas validações
    int quantidade = 0;
    int telefonevalido;
    int emailExistente = 0;
    int temMaiuscula;
    int temEspecial;

    // Ponteiro utilizado para abrir, ler e salvar arquivos
    FILE *cadastrocliente;


    // ==================== GERAR ID ====================

    // Abre o arquivo para contar os clientes já cadastrados
    cadastrocliente = fopen("../../Dados/cliente.dat", "rb");

    if (cadastrocliente != NULL) {

        // Lê cada registro e aumenta a quantidade de clientes
        while (fread(&clientearquivo, sizeof(struct Cliente), 1, cadastrocliente) == 1) {
            quantidade++;
        }

        // Fecha o arquivo após a leitura
        fclose(cadastrocliente);
    }
```

```c
    // Gera o ID do novo cliente
    cliente.id = quantidade + 1;


    // ==================== CADASTRAR NOME ====================

    printf("Digite seu nome\n");

    // Permite digitar nomes com espaços, respeitando o limite
    scanf(" %49[^\n]", cliente.nome);


    // ==================== VALIDAR TELEFONE ====================

    do {
        printf("Digite seu telefone\n");
        scanf("%19s", cliente.telefone);

        // Inicialmente considera o telefone válido
        telefonevalido = 1;

        // Percorre cada caractere digitado
        for (int i = 0; i < strlen(cliente.telefone); i++) {

            // Se encontrar algo diferente de 0 a 9, invalida
            if (cliente.telefone[i] < '0' || cliente.telefone[i] > '9') {
                telefonevalido = 0;
            }
        }

        // Repete se não tiver 10 ou 11 dígitos numéricos
    } while (strlen(cliente.telefone) < 10 ||
             strlen(cliente.telefone) > 11 ||
             telefonevalido == 0);
```

```c
    // ================== VALIDAR EMAIL ==================

    do {

        // Verifica se o email contém @ e ponto
        do {
            printf("Digite seu email\n");
            scanf("%99s", cliente.email);

        } while (strchr(cliente.email, '@') == NULL ||
                 strchr(cliente.email, '.') == NULL);

        // Abre o arquivo para procurar emails já cadastrados
        cadastrocliente = fopen("../../Dados/cliente.dat", "rb");

        // 0 = email disponível | 1 = email já cadastrado
        emailExistente = 0;

        if (cadastrocliente != NULL) {

            // Percorre os clientes salvos no arquivo
            while (fread(&clientearquivo, sizeof(struct Cliente), 1, cadastrocliente) == 1) {

                // strcmp retorna 0 quando os emails são iguais
                if (strcmp(cliente.email, clientearquivo.email) == 0) {
                    emailExistente = 1;
                }
            }

            fclose(cadastrocliente);

            // Informa se o email já estiver cadastrado
            if (emailExistente == 1) {
                printf("Email ja cadastrado!\n");
            }
        }

        // Solicita outro email enquanto houver duplicidade
    } while (emailExistente == 1);


```

```c
    // ================== VALIDAR SENHA ==================

    // Apresenta os requisitos antes de solicitar a senha
    printf("\nSua senha deve conter:\n");
    printf("- No minimo 6 caracteres\n");
    printf("- Pelo menos 1 letra maiuscula\n");
    printf("- Pelo menos 1 caractere especial (@, #, !, $)\n");

    do {
        printf("\nDigite uma senha\n");
        scanf("%49s", cliente.senha);

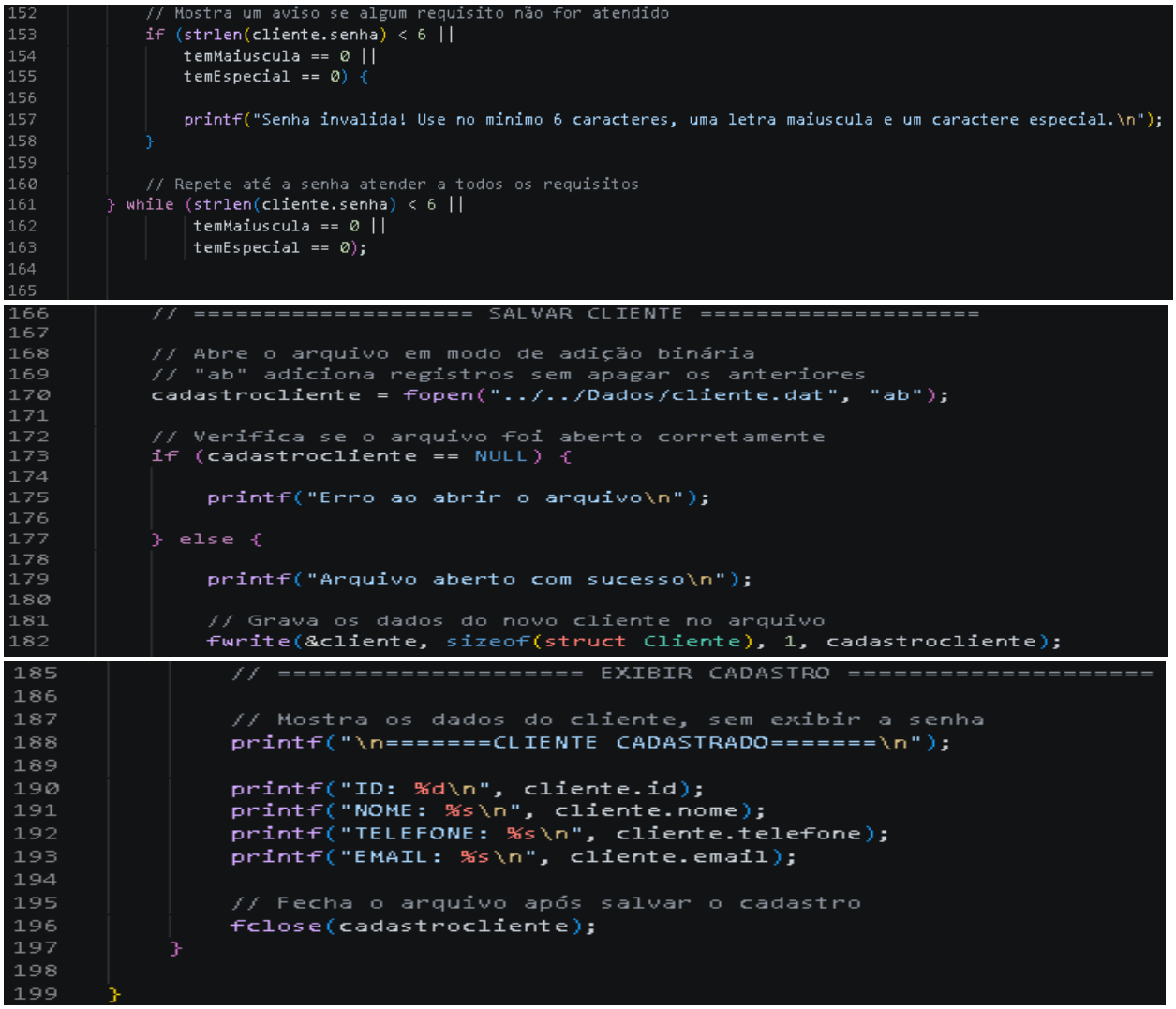
        // Reinicia as verificações a cada tentativa
        temMaiuscula = 0;
        temEspecial = 0;

        // Percorre os caracteres da senha
        for (int i = 0; i < strlen(cliente.senha); i++) {

            // Verifica se existe pelo menos uma letra maiúscula
            if (cliente.senha[i] >= 'A' && cliente.senha[i] <= 'Z') {
                temMaiuscula = 1;
            }

            // Verifica se existe pelo menos um caractere especial
            if ((cliente.senha[i] >= '!' && cliente.senha[i] <= '/') ||
                (cliente.senha[i] >= ':' && cliente.senha[i] <= '@') ||
                (cliente.senha[i] >= '[' && cliente.senha[i] <= '`') ||
                (cliente.senha[i] >= '{' && cliente.senha[i] <= '~')) {

                temEspecial = 1;
            }
        }
```



```
152         // Mostra um aviso se algum requisito não for atendido
153         if (strlen(cliente.senha) < 6 ||
154             temMaiuscula == 0 ||
155             temEspecial == 0) {
156 
157             printf("Senha invalida! Use no minimo 6 caracteres, uma letra maiuscula e um caractere especial.\n");
158         }
159 
160         // Repete até a senha atender a todos os requisitos
161     } while (strlen(cliente.senha) < 6 ||
162              temMaiuscula == 0 ||
163              temEspecial == 0);
164 
165 
166         // ==================== SALVAR CLIENTE ====================
167 
168         // Abre o arquivo em modo de adição binária
169         // "ab" adiciona registros sem apagar os anteriores
170         cadastrocliente = fopen("../../Dados/cliente.dat", "ab");
171 
172         // Verifica se o arquivo foi aberto corretamente
173         if (cadastrocliente == NULL) {
174 
175             printf("Erro ao abrir o arquivo\n");
176 
177         } else {
178 
179             printf("Arquivo aberto com sucesso\n");
180 
181             // Grava os dados do novo cliente no arquivo
182             fwrite(&cliente, sizeof(struct Cliente), 1, cadastrocliente);
185             // ==================== EXIBIR CADASTRO ====================
186 
187             // Mostra os dados do cliente, sem exibir a senha
188             printf("\n=======CLIENTE CADASTRADO=======\n");
189 
190             printf("ID: %d\n", cliente.id);
191             printf("NOME: %s\n", cliente.nome);
192             printf("TELEFONE: %s\n", cliente.telefone);
193             printf("EMAIL: %s\n", cliente.email);
194 
195             // Fecha o arquivo após salvar o cadastro
196             fclose(cadastrocliente);
197         }
198 
199     }
```

Módulo de persistência binária e rotina de login

Arquivo Login.c :

```c
#include <stdio.h>
#include <string.h>
#include "../Cliente/Cliente.h"

int fazerLogin() {

    char email[100];
    char senha[50];
    int loginValido = 0;

    struct Cliente clienteArquivo;

    FILE *arquivo;
    arquivo = fopen("../../Dados/cliente.dat", "rb");

    if (arquivo == NULL) {
        printf("Erro ao abrir o arquivo de clientes!\n");
        return -1;
    }

    printf("Arquivo aberto com sucesso!\n");

    printf("Digite seu email.\n");
    scanf("%99s", email);

    printf("Digite sua senha.\n");
    scanf("%49s", senha);

    while (fread(&clienteArquivo, sizeof(struct Cliente), 1, arquivo) == 1) {

            if(strcmp(email, clienteArquivo.email) == 0 && strcmp(senha, clienteArquivo.senha) == 0) {

                loginValido = 1;

                printf("Login realizado com sucesso!\n");
                printf("Bem-vindo, %s!\n", clienteArquivo.nome);
                fclose(arquivo);
                return clienteArquivo.id;
            }
    }
    fclose(arquivo);

    if (loginValido == 0) {
        printf("Email ou senha incorretos!\n");
    }

    return -1;
}
```

Módulo principal e gerenciador de rotas e menus

Arquivo main.c :

```c
#include <stdio.h>
void cadastrarCliente(void);
int fazerLogin(void);
void menuCliente(int idCliente);

int main() {

    int opcao = 0;
    int idCliente = -1;

    while (opcao != 3) {

        printf("\n===== SISTEMA BARBEARIA =====\n");

        printf("1 - Cadastrar cliente\n");
        printf("2 - Fazer login\n");
        printf("3 - Sair\n");

        printf("Escolha uma opcao: ");
        scanf("%d", &opcao);

        switch (opcao){
            case 1:
                cadastrarCliente();
                break;

            case 2:
                idCliente = fazerLogin();
                if (idCliente > 0) {
                    printf("Cliente logado com ID: %d\n", idCliente);
                    menuCliente(idCliente);
                    idCliente = -1;
                }
                break;

            case 3:
                printf("Saindo...\n");
                break;

        default:
            printf("Opcao invalida!\n");
        }
    }

    return 0;
}
```

Arquivo MenuCliente.c :

```c
#include <stdio.h>

void agendarHorario(int idCliente);
void menuCliente(int idCliente) {

    int opcao = 0;

    while(opcao != 4) {

        printf("\n===== AREA DO CLIENTE =====\n");
        printf("Cliente ID: %d\n", idCliente);

        printf("1 - Agendar horario\n");
        printf("2 - Meus agendamento\n");
        printf("3 - Cancelar agendamento\n");
        printf("4 - Sair da conta\n");

        printf("Escolha uma opcao: ");
        scanf("%d", &opcao);

        switch (opcao) {

            case 1:
                agendarHorario(idCliente);
                break;

            case 2:
                printf("Meus agendamentos selecionado!\n");
                break;

            case 3:
                printf("Cancelar agendamento selecionado!\n");
                break;

            case 4:
                printf("Saindo...\n");
                break;

        default:
            printf("Opcao invalida\n");


        }
    }
}
```